

Pattern Recognition and Machine Learning (EE602L)

Lab 3: Logistic Regression — Parameter Estimation and Regularization

Instructions:

- Use Google Colab.
- Submit one well-commented notebook (Lab3-RollNum.ipynb) along with a PDF report (Lab3-RollNum.pdf) containing the experimental outputs, analysis of the results, and conclusions.
- Wrong file names or missing files won't be graded.
- **Deadline: 31/08/2026 11.59PM. Late submissions lose 10% per day.**
- This is an individual assignment. Please follow the Institute's Academic Code of Honor.

Dataset: Breast Cancer Wisconsin (Diagnostic) dataset

Available directly through scikit-learn, with no download required:

```
from sklearn.datasets import load_breast_cancer
```

Original source (UCI Machine Learning Repository):

<https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic>

Data Preparation (common to both parts):

Before starting either part, load the dataset with `load_breast_cancer()` and get it ready for modelling. Inspect the shape, feature names, class balance and summary statistics; verify there are no missing values or duplicate records (and handle them if you do find any), and detect outliers in a couple of features using a clearly stated and consistently applied criterion, e.g. the IQR rule. Visualize the data before you model it: at minimum, look at the class balance and how one or two features separate the two classes. Then implement a 70:15:15 train-validation-test split on your own (NumPy/Pandas only, no `sklearn.model_selection`) using a fixed random seed of 42 so the split is reproducible, check that the split sizes are correct and that no row appears in more than one set, and standardize the numerical features using statistics computed from the training set only; this matters even more here than in Lab 2, since the regularization penalty in Part (b) is scale-sensitive.

Carry this same cleaned, split and scaled dataset through both Part (a) and Part (b) below. You should not need to repeat any of this work.

Part (a): Logistic Regression via Gradient Descent

Add an intercept term and define the model through the sigmoid link $\sigma(z) = \frac{1}{1+e^{-z}}$ and predicted probability $\hat{\mathbf{y}} = \sigma(X\theta)$ (y_i the true label, $\hat{y}_i$ the corresponding predicted probability, and $\hat{\mathbf{y}}$ without a subscript the full n -vector of predicted probabilities), optimized against the cross-entropy cost $J(\theta) = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$ (clip $\hat{y}_i$ away from 0 and 1 only when evaluating these logs, for numerical stability — this clipping applies solely inside the log-loss computation, not as a permanent change to the sigmoid output), with gradient $\nabla_{\theta} J(\theta) = \frac{1}{n} X^T (\hat{\mathbf{y}} - \mathbf{y})$ (X the $n \times (d+1)$ design matrix with the intercept column, $\hat{\mathbf{y}}$ and $\mathbf{y}$ stacked as n -vectors). Implement batch gradient descent from scratch using $\theta \leftarrow \theta - \eta \nabla_{\theta} J(\theta)$ (η the learning rate); train it with at least three learning rates, plot the training-cost curves, and pick the best setting using validation log-loss. Classify a sample as the positive class when its predicted probability $\hat{y}_i \geq 0.5$, otherwise as the negative class; use this thresholded prediction for accuracy, precision, recall and F1 (log-loss and ROC-AUC use the predicted probabilities directly). Report the learned intercept and coefficients with their feature names, and compute accuracy, precision, recall, F1-score and log-loss manually, using $\text{Precision} = \frac{TP}{TP+FP}$, $\text{Recall} = \frac{TP}{TP+FN}$ and $F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$ (you may use `sklearn.metrics` only for ROC-AUC). Evaluate the model on the test set, report test accuracy, precision, recall, F1, log-loss and ROC-AUC, plot the ROC curve, and briefly comment on prediction quality.

Part (b): Logistic Regression vs. L2 vs. L1 vs. Elastic Net Regularization

Using the same prepared features, fit a plain (unregularized) Logistic Regression baseline (reuse your Part (a) model). Examine the feature correlation matrix, and separately report the condition number of the standardized training-feature design matrix (excluding the intercept column), to find at least two highly correlated feature groups (e.g. mean radius, mean perimeter and mean area, or mean concavity and mean concave points), and comment on what this does to coefficient stability. Implement L2-regularized (Ridge-style) logistic regression using $J_{L2}(\theta) = J(\theta) + \frac{\alpha}{2n} \theta^T R \theta$, $R =$

$\text{diag}(0, 1, \dots, 1)$ (intercept unregularized), gradient $\nabla_{\theta} J_{L2}(\theta) = \nabla_{\theta} J(\theta) + \frac{\alpha}{n} R\theta$; sweep at least five $\alpha > 0$ values plus $\alpha = 0$, selecting the best via validation log-loss. Separately fit L1-regularized and Elastic-Net-regularized logistic regression via `sklearn.linear_model.LogisticRegression` (`penalty='l1'`, `solver='liblinear'`; and `penalty='elasticnet'`, `solver='saga'`, with a chosen `l1_ratio` ρ and `max_iter=5000`, or another sufficiently large value, to ensure convergence) over a range of C values (scikit-learn's own inverse-regularization-strength parameter, distinct from the α above), each selected via validation log-loss; the elastic-net penalty on the non-intercept coefficients is $J_{EN}(\theta) = J(\theta) + \frac{\alpha}{n} \left[\rho \|\theta_{1:}\|_1 + \frac{1-\rho}{2} \|\theta_{1:}\|_2^2 \right]$, with $\rho=1$ recovering L1 and $\rho=0$ recovering L2 (excluding the intercept θ_0 , matching scikit-learn's default of leaving the intercept unregularized). C and α are different parameters under different objective normalizations, so do not attempt to convert one into the other; instead, discuss qualitatively how larger C means weaker regularization while larger α means stronger regularization, and how that shows up in the coefficient paths, as in Lab 2. Report how many coefficients the L1 and Elastic-Net models drive to zero (treating a coefficient θ_j as zero if $|\theta_j| < 10^{-6}$, with the intercept excluded from this count), and briefly explain why L1-type penalties induce sparsity while L2 does not. Plot coefficient magnitude against regularization strength for L2, L1 and Elastic Net; because L2 uses α while L1/Elastic Net use scikit-learn's C , do not place α and C on the same numerical x-axis as if they were directly comparable — use separate panels/axes, or otherwise clearly indicate the different parameterizations and their direction of regularization strength (larger $\alpha \rightarrow$ stronger L2 regularization; smaller $C \rightarrow$ stronger L1/Elastic Net regularization) — and describe how the paths differ. Evaluate the baseline and the best L2, L1 and Elastic-Net models on the test set in one comparison table (test accuracy, precision, recall, F1, log-loss, ROC-AUC, coefficient L2 norm excluding the intercept, and number of non-zero coefficients). Close with a short discussion of when you would prefer each approach, grounded in your results, including one real-world scenario where Elastic Net's combined behaviour is especially useful.